

NP-Completeness Foundations and Exercises

Compact Question–Answer Notes: Exercises 34.1–34.5

Exercises 34.1 – Polynomial Time

Question 34.1-1

Show that LONGEST-PATH-LENGTH can be solved in polynomial time iff the decision problem LONGEST-PATH is in **P**.

Answer

Optimization $\Rightarrow$ **decision**: compute the optimum length ℓ and accept iff $\ell \geq k$.

Decision $\Rightarrow$ **optimization**: the answer lies in $[0, |E|]$. Binary-search k using the decision algorithm. This needs $\mathcal{O}(\log |E|)$ polynomial-time calls. Hence the two versions are polynomially equivalent.

Question 34.1-2

Define the longest-simple-cycle optimization problem, a related decision problem, and its language.

Answer

Optimization: return a simple cycle of maximum length in G .

Decision: does G contain a simple cycle of length at least k ?

$$L = \{\langle G, k \rangle \mid G \text{ has a simple cycle with at least } k \text{ edges}\}.$$

Question 34.1-3

Give binary encodings of directed graphs using adjacency matrices and adjacency lists, and show the encodings are polynomially related.

Answer

Number vertices $0, \dots, n-1$.

Matrix: encode n , then the n^2 adjacency bits in row-major order. Size $\Theta(n^2)$.

List: encode n , then each vertex's outgoing-neighbor indices with unambiguous delimiters. Size is polynomial in $n + |E|$.

List $\rightarrow$ matrix: scan lists and set matrix entries. Matrix $\rightarrow$ list: scan all n^2 entries and output every 1-entry. Both conversions are polynomial time.

Question 34.1-4

Is the standard dynamic-programming algorithm for 0-1 knapsack polynomial time? Explain.

Answer

No. Its time is $\Theta(nW)$. If W uses b binary bits, then $b = \Theta(\log W)$ and $W = \Theta(2^b)$. Thus

$$\Theta(nW) = \Theta(n2^b),$$

which is exponential in the input length. It is **pseudo-polynomial**.

Question 34.1-5

Show that a constant number of polynomial-time subroutine calls preserves polynomial time, but a polynomial number of such calls may lead to exponential time.

Answer

A constant number of calls gives only a constant sum/composition of polynomials, so the total remains polynomial.

For the second part, let a polynomial-time subroutine square its integer input. Starting with an N -bit number

and feeding each output to the next call:

$$x, x^2, x^4, \dots, x^{2^N}.$$

After only N calls, the last output has $\Theta(2^N N)$ bits. Writing it already takes exponential time. The danger is **growing intermediate input size**.

Question 34.1-6

Show that $\mathbf{P}$ is closed under union, intersection, concatenation, complement, and Kleene star.

Answer

Let A_1, A_2 decide L_1, L_2 in polynomial time.

- $L_1 \cup L_2$: accept if either decider accepts.
- $L_1 \cap L_2$: accept if both accept.
- L_1^c : run A_1 and flip the answer.
- $L_1 L_2$: try all $n + 1$ splits $x = yz$; accept if $y \in L_1$ and $z \in L_2$ for some split.
- L_1^* : dynamic programming over prefix split points; at most $\mathcal{O}(n^2)$ substring checks.

All use polynomially many polynomial-time operations, so each language is in $\mathbf{P}$.

Exercises 34.2 – NP and Certificates

Question 34.2-1

Prove that GRAPH-ISOMORPHISM $\in \mathbf{NP}$ by describing a polynomial-time verifier.

Answer

Certificate: a proposed bijection $\pi : V(G_1) \rightarrow V(G_2)$.

Verify that π is a bijection and, for every u, v ,

$$(u, v) \in E(G_1) \iff (\pi(u), \pi(v)) \in E(G_2).$$

There are $\mathcal{O}(|V|^2)$ pairs, so verification is polynomial. Hence GRAPH-ISOMORPHISM $\in \mathbf{NP}$.

Question 34.2-2

Prove that an undirected bipartite graph with an odd number of vertices is nonhamiltonian.

Answer

Every cycle in a bipartite graph alternates between the two parts, so every cycle has even length. A Hamiltonian cycle has length $|V|$. If $|V|$ is odd, such a cycle is impossible.

Question 34.2-3

Show that if HAM-CYCLE $\in \mathbf{P}$, then listing the vertices of a Hamiltonian cycle in order is also polynomial-time solvable.

Answer

Let D be the polynomial-time Hamiltonian-cycle decider.

1. If $D(G) = \text{No}$, stop.
2. Set $H = G$. For each edge e , if $D(H - e) = \text{Yes}$, delete e .
3. The remaining graph is exactly a Hamiltonian cycle; traverse it to list the vertices.

Why? Any remaining edge not on a Hamiltonian cycle could still be deleted, contradiction. At most $|E| + 1$ calls are made, so the total time is polynomial.

Question 34.2-4

Prove that $\mathbf{NP}$ is closed under union, intersection, concatenation, and Kleene star. Discuss complement.

Answer

Use polynomial certificates:

- Union: certificate (b, y) says which language to verify.
- Intersection: certificate (y_1, y_2) verifies membership in both.
- Concatenation: certificate gives split $x = x_1x_2$ plus certificates for both pieces.
- Kleene star: certificate gives a decomposition $x = x_1 \cdots x_t$ and a certificate for every piece; $t \leq |x|$.

All certificates and checks are polynomial. Complement closure is unknown; it is exactly the question whether $\mathbf{NP} = \mathbf{coNP}$.

Question 34.2-5

Show that every language in $\mathbf{NP}$ can be decided in time $2^{\mathcal{O}(n^k)}$ for some constant k .

Answer

If certificates have length at most $p(n) = \mathcal{O}(n^c)$, enumerate all possible certificates and run the verifier. There are

$$2^{p(n)} = 2^{\mathcal{O}(n^c)}$$

candidates. The verifier's polynomial cost is absorbed into the exponential bound. Therefore the language is decidable in $2^{\mathcal{O}(n^k)}$ time for some k .

Question 34.2-6

Show that the Hamiltonian-path language HAM-PATH belongs to $\mathbf{NP}$.

Answer

Certificate: $(v_1, \dots, v_n)$. Verify:

1. $v_1 = u$ and $v_n = v$;
2. every vertex appears exactly once;
3. $(v_i, v_{i+1}) \in E$ for all i .

All checks are polynomial, so HAM-PATH $\in \mathbf{NP}$.

Question 34.2-7

Show that Hamiltonian path is polynomial-time solvable on directed acyclic graphs (DAGs).

Answer

Compute a topological order $v_1, \dots, v_n$. Accept iff $v_1 = u$, $v_n = v$, and every edge (v_i, v_{i+1}) exists.

If a DAG has a Hamiltonian path, that path forces every topological ordering to list the vertices in that path order. Thus the test is correct. Topological sorting takes $\mathcal{O}(|V| + |E|)$ time.

Question 34.2-8

Show that TAUTOLOGY belongs to $\mathbf{coNP}$.

Answer

A formula is *not* a tautology iff some assignment makes it false. That assignment is a polynomial-size certificate for TAUTOLOGY. Evaluating the formula is polynomial time. Hence

$$\overline{\text{TAUTOLOGY}} \in \mathbf{NP} \quad \Rightarrow \quad \text{TAUTOLOGY} \in \mathbf{coNP}.$$

Question 34.2-9

Prove that $\mathbf{P} \subseteq \mathbf{coNP}$.

Answer

Let $L \in \mathbf{P}$. A polynomial-time decider for L can decide $\bar{L}$ by flipping its answer. Thus $\bar{L} \in \mathbf{P} \subseteq \mathbf{NP}$. By definition, $L \in \mathbf{coNP}$. Therefore

$$\mathbf{P} \subseteq \mathbf{coNP}.$$

Question 34.2-10

Prove that if $\mathbf{NP} \neq \mathbf{coNP}$, then $\mathbf{P} \neq \mathbf{NP}$.

Answer

Use the contrapositive. Assume $\mathbf{P} = \mathbf{NP}$. Since $\mathbf{P}$ is closed under complement, every $L \in \mathbf{NP} = \mathbf{P}$ has $\bar{L} \in \mathbf{P} = \mathbf{NP}$. Therefore $\mathbf{NP}$ is closed under complement, so $\mathbf{NP} = \mathbf{coNP}$. Hence

$$\mathbf{NP} \neq \mathbf{coNP} \Rightarrow \mathbf{P} \neq \mathbf{NP}.$$

Question 34.2-11

Let G be connected with at least 3 vertices. In G^3 , vertices at distance at most 3 in G are adjacent. Prove that G^3 is Hamiltonian.

Answer

Take a spanning tree T of G ; it is enough to show T^3 is Hamiltonian because $T^3 \subseteq G^3$. Root T and perform DFS. Record each odd-depth vertex when first visited and each even-depth vertex when last visited. This records every vertex exactly once. Consecutive recorded vertices are at tree-distance at most 3, and the last and first are also within distance 3. Therefore all consecutive pairs are edges in T^3 , forming a Hamiltonian cycle. Hence G^3 is Hamiltonian.

Exercises 34.3 – Polynomial Reductions

Question 34.3-1

Verify that the circuit in Figure 34.8(b) is unsatisfiable.

Answer

After canceling double negations, one conjunct contains

$$(x_1 \vee x_2) \wedge x_3,$$

which forces $x_3 = 1$, while another contains

$$x_1 \wedge \neg x_3 \wedge x_2,$$

which forces $x_3 = 0$. Since the whole output is an AND, both would have to hold. Impossible. Therefore the circuit is unsatisfiable.

Question 34.3-2

Prove that polynomial-time reductions are transitive: if $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$, then $L_1 \leq_p L_3$.

Answer

Let f_1 reduce L_1 to L_2 and f_2 reduce L_2 to L_3 . Define

$$g(x) = f_2(f_1(x)).$$

Polynomial-time functions compose, and

$$x \in L_1 \iff f_1(x) \in L_2 \iff f_2(f_1(x)) \in L_3.$$

Hence g is a polynomial-time reduction and $L_1 \leq_p L_3$.

Question 34.3-3

Prove $L \leq_p \bar{L}$ iff $\bar{L} \leq_p L$.

Answer

Suppose f satisfies

$$x \in L \iff f(x) \in \bar{L}.$$

Complement both sides:

$$x \in \bar{L} \iff f(x) \in L.$$

So the same f reduces $\bar{L}$ to L . The reverse direction is identical.

Question 34.3-4

Could a satisfying input assignment be used as the certificate for Circuit-SAT? Which certificate gives the easier proof?

Answer

Yes. Given the input assignment, evaluate gates in topological order and accept iff the output is 1; this is polynomial time.

However, the textbook certificate containing **all wire values** makes the proof easier because the verifier only checks local gate consistency. The input-only certificate is shorter but requires evaluating the circuit.

Question 34.3-5

Where is the contiguous polynomial-size working-storage assumption used in Lemma 34.6, and why is it harmless?

Answer

It is used to ensure that each machine configuration has only polynomially many bits, so the simulation circuit is polynomial size. A polynomial-time computation can access only polynomially many memory cells. If those cells are scattered, store them as a polynomial-size table of address-value pairs. Thus the assumption causes no loss of generality.

Question 34.3-6

Show that $\emptyset$ and Σ^* are the only languages in $\mathbf{P}$ that are not complete for $\mathbf{P}$ under polynomial reductions.

Answer

$\emptyset$ and Σ^* cannot be complete: no reduction can map an always-yes language to an always-no language with membership preserved, or vice versa.

Now let $L \in \mathbf{P}$ be nontrivial. Choose fixed $a \in L$ and $b \notin L$. For any $L' \in \mathbf{P}$ with decider A' , define

$$f(x) = \begin{cases} a, & A'(x) = \text{YES}, \\ b, & A'(x) = \text{NO}. \end{cases}$$

Then $x \in L' \iff f(x) \in L$, and f is polynomial time. So every nontrivial $L \in \mathbf{P}$ is $\mathbf{P}$ -complete.

Question 34.3-7

Show the complement relationship between NP-completeness and coNP-completeness.

Answer

If L is $\mathbf{NP}$ -complete, then $\bar{L} \in \mathbf{coNP}$. For any $S \in \mathbf{coNP}$, $\bar{S} \in \mathbf{NP}$, so

$$\bar{S} \leq_p L.$$

Using the same reduction and complementing membership gives

$$S \leq_p \bar{L}.$$

Hence $\bar{L}$ is $\mathbf{coNP}$ -complete. The converse follows by the same argument on complements.

Question 34.3-8

Why is it valid that the reduction proving CIRCUIT-SAT NP-hard uses a verifier A that depends on the language $L \in \mathbf{NP}$?

Answer

NP-hardness requires: *for each fixed* $L \in \mathbf{NP}$, there exists a polynomial-time reduction $L \leq_p \text{CIRCUIT-SAT}$. Once L is fixed, its polynomial-time verifier A is fixed too. The reduction may use the fixed code of A to build a circuit simulating $A(x, y)$. Therefore the dependence on A is valid.

Exercises 34.4 – SAT and CNF

Question 34.4-1

Give a size- n circuit whose straightforward conversion to a formula produces exponential size.

Answer

Let $g_0 = x_1 \wedge x_2$ and, for $i = 1, \dots, n$,

$$g_i = g_{i-1} \wedge g_{i-1}.$$

The circuit has $\Theta(n)$ gates, but direct formula expansion gives $F_i = F_{i-1} \wedge F_{i-1}$. If $s_i = |F_i|$,

$$s_i = 2s_{i-1} + 1 = \Theta(2^i).$$

So linear circuit size can expand to exponential formula size.

Question 34.4-2

Show the 3-CNF formula obtained by applying the method of Theorem 34.10 to

$$\phi = ((x_1 \Rightarrow x_2) \vee \neg((\neg x_1 \Leftrightarrow x_3) \vee x_4)) \wedge \neg x_2.$$

Answer

Introduce variables for subformulas:

$$\begin{aligned} \phi' = & y_1 \wedge (y_1 \Leftrightarrow (y_2 \wedge \neg x_2)) \wedge (y_2 \Leftrightarrow (y_3 \vee y_4)) \\ & \wedge (y_3 \Leftrightarrow (x_1 \Rightarrow x_2)) \wedge (y_4 \Leftrightarrow \neg y_5) \\ & \wedge (y_5 \Leftrightarrow (y_6 \vee x_4)) \wedge (y_6 \Leftrightarrow (\neg x_1 \Leftrightarrow x_3)). \end{aligned}$$

The final 3-CNF is $\psi = P_1 \wedge E_1 \wedge \dots \wedge E_6$, where p, q are fresh variables:

$$\begin{aligned} P_1 &= (y_1 \vee p \vee q)(y_1 \vee p \vee \neg q)(y_1 \vee \neg p \vee q)(y_1 \vee \neg p \vee \neg q), \\ E_1 &= (\neg y_1 \vee \neg y_2 \vee \neg x_2)(\neg y_1 \vee y_2 \vee \neg x_2)(\neg y_1 \vee y_2 \vee x_2)(y_1 \vee \neg y_2 \vee x_2), \\ E_2 &= (\neg y_2 \vee y_3 \vee y_4)(y_2 \vee \neg y_3 \vee \neg y_4)(y_2 \vee \neg y_3 \vee y_4)(y_2 \vee y_3 \vee \neg y_4), \\ E_3 &= (y_3 \vee x_1 \vee x_2)(y_3 \vee x_1 \vee \neg x_2)(y_3 \vee \neg x_1 \vee \neg x_2)(\neg y_3 \vee \neg x_1 \vee x_2), \\ E_4 &= (y_4 \vee y_5 \vee p)(y_4 \vee y_5 \vee \neg p)(\neg y_4 \vee \neg y_5 \vee p)(\neg y_4 \vee \neg y_5 \vee \neg p), \\ E_5 &= (\neg y_5 \vee y_6 \vee x_4)(y_5 \vee \neg y_6 \vee \neg x_4)(y_5 \vee \neg y_6 \vee x_4)(y_5 \vee y_6 \vee \neg x_4), \\ E_6 &= (\neg y_6 \vee \neg x_1 \vee \neg x_3)(\neg y_6 \vee x_1 \vee x_3)(y_6 \vee \neg x_1 \vee x_3)(y_6 \vee x_1 \vee \neg x_3). \end{aligned}$$

Here juxtaposition between parenthesized clauses means conjunction.

Question 34.4-3

Why does using a truth table for the whole formula fail to give a polynomial-time reduction from SAT to 3-CNF-SAT?

Answer

A formula with n variables has 2^n assignments. Writing its full truth table already needs $\Omega(2^n)$ time and space, and the resulting CNF can have exponentially many clauses. Therefore the construction is not polynomial time.

Question 34.4-4

Show that deciding whether a Boolean formula is a tautology is complete for **coNP**.

Answer

In coNP: a certificate for “not a tautology” is an assignment making the formula false.

coNP-hard: since UNSAT is coNP-complete, reduce

$$\phi \mapsto \neg\phi.$$

Then

$$\phi \in \text{UNSAT} \iff \neg\phi \in \text{TAUTOLOGY}.$$

The map is polynomial, so $\text{UNSAT} \leq_p \text{TAUTOLOGY}$. Therefore TAUTOLOGY is **coNP**-complete.

Question 34.4-5

Show that satisfiability of Boolean formulas in disjunctive normal form (DNF) is polynomial-time solvable.

Answer

For

$$T_1 \vee T_2 \vee \cdots \vee T_m,$$

a term T_i is satisfiable iff it does **not** contain both x and $\neg x$ for any variable. Scan every term; accept if at least one term has no conflict. This is polynomial time (linear with suitable marker arrays).

Question 34.4-6

Assume SAT can be decided in polynomial time. Show how to find an actual satisfying assignment in polynomial time.

Answer

Run the SAT decider first. If unsatisfiable, stop. Otherwise fix variables one by one. For each x_i , temporarily set $x_i = 1$ and ask whether the restricted formula remains satisfiable. If yes, keep $x_i = 1$; otherwise set $x_i = 0$. After n variables, the chosen values form a satisfying assignment. This uses at most $n + 1$ polynomial-time SAT calls.

Question 34.4-7

Show that 2-CNF-SAT is in **P** and give an efficient algorithm.

Answer

Build the implication graph. For each clause $(a \vee b)$ add

$$\neg a \rightarrow b, \quad \neg b \rightarrow a.$$

Compute strongly connected components. The formula is satisfiable iff no variable x_i and $\neg x_i$ lie in the same SCC. If they do, each implies the other, impossible under a satisfying assignment. Otherwise an assignment follows from a topological order of the SCC DAG. With $2n$ vertices and $2m$ edges, SCCs take

$$\mathcal{O}(n + m).$$

Thus 2-CNF-SAT $\in \mathbf{P}$.

Exercises 34.5 – NP-Complete Problems

Question 34.5-1

The subgraph-isomorphism problem asks whether G_1 is isomorphic to a subgraph of G_2 . Show that SUBGRAPH-ISOMORPHISM is NP-complete.

Answer

In NP: certificate = injective map $f : V(G_1) \rightarrow V(G_2)$; verify all edges are preserved.

NP-hard: reduce $\text{CLIQUE} \leq_p \text{SUBGRAPH-ISOMORPHISM}$. Given (G, k) , set

$$G_1 = K_k, \quad G_2 = G.$$

Then K_k is isomorphic to a subgraph of G iff G has a k -clique. The construction is polynomial. Hence $\text{SUBGRAPH-ISOMORPHISM}$ is **NP**-complete.

Question 34.5-2

Given integer A, b , 0-1 integer programming asks whether some $x \in \{0, 1\}^n$ satisfies $Ax \leq b$. Prove it is NP-complete.

Answer

In NP: verify a proposed 0-1 vector by computing Ax .

Reduction from 3-Cnf-Sat: use one 0-1 variable per Boolean variable. For clause C_j , with positive indices P_j and negated indices N_j , require

$$\sum_{i \in P_j} x_i + \sum_{i \in N_j} (1 - x_i) \geq 1,$$

or equivalently

$$-\sum_{i \in P_j} x_i + \sum_{i \in N_j} x_i \leq |N_j| - 1.$$

One inequality per clause gives a polynomial construction, and the system is feasible iff the formula is satisfiable. Therefore 0-1-IP is **NP**-complete.

Question 34.5-3

Assuming 0-1 integer programming is NP-hard, show that integer linear programming is NP-complete.

Answer

Given $Ax \leq b$ with $x_i \in \{0, 1\}$, use integer variables and add

$$x_i \leq 1, \quad -x_i \leq 0$$

for every i . Integer solutions are then forced to $x_i \in \{0, 1\}$. Thus 0-1-IP reduces polynomially to ILP, so ILP is NP-hard. A feasible integer solution of polynomial bit length can be verified by substitution, so ILP is in **NP**. Hence it is **NP**-complete.

Question 34.5-4

Show how to solve SUBSET-SUM in polynomial time when the target t is written in unary.

Answer

Use DP: $dp[i, s] = \text{true}$ iff the first i numbers can form sum s .

$$dp[i, s] = dp[i-1, s] \vee (s \geq a_i \wedge dp[i-1, s-a_i]).$$

Initialize $dp[0, 0] = \text{true}$. The answer is $dp[n, t]$. Time is $\mathcal{O}(nt)$. When t is unary, t is at most linear in the input length, so this is polynomial time.

Question 34.5-5

Set-Partition asks whether a set of numbers can be divided into two parts having equal sum. Show that SET-PARTITION is NP-complete.

Answer

In NP: verify one side of the partition by summing both sides.

Reduce from Subset-Sum: for instance (S, t) let $x = \sum_{s \in S} s$ and construct

$$S' = S \cup \{x+t, 2x-t\}.$$

Total sum is $4x$, so each partition side must sum to $2x$. If $B \subseteq S$ sums to t , then $B \cup \{2x - t\}$ sums to $2x$. Conversely, in any equal partition the two added numbers must be on opposite sides; the side containing $2x - t$ needs original elements totaling t . Thus

$$(S, t) \in \text{SUBSET-SUM} \iff S' \in \text{SET-PARTITION}.$$

Hence SET-PARTITION is **NP**-complete.

Question 34.5-6

Show that the Hamiltonian-path problem is NP-complete.

Answer

In NP: verify a path listing every vertex once.

Reduce from Ham-Cycle: choose a vertex v in G , split it into v_{in} and v_{out} , connect both to all former neighbors of v , and attach leaves s to v_{in} and t to v_{out} . Then

$$G \text{ has a Hamiltonian cycle} \iff G' \text{ has a Hamiltonian path } s \rightsquigarrow t.$$

A cycle through v opens into the path; any Hamiltonian path in G' must start/end at the two leaves and closes back into a cycle when the split vertices are merged. Construction is polynomial, so HAM-PATH is **NP**-complete.

Question 34.5-7

Formulate a decision version of longest-simple-cycle and show that it is NP-complete.

Answer

Define

$$\text{LONG-SIMPLE-CYCLE} = \{\langle G, k \rangle \mid G \text{ has a simple cycle of length at least } k\}.$$

In NP: verify the claimed cycle.

NP-hard: reduce HAM-CYCLE $\leq_p$ LONG-SIMPLE-CYCLE by mapping

$$G \mapsto \langle G, |V| \rangle.$$

A simple cycle of length at least $|V|$ uses every vertex, so it is Hamiltonian. Hence LONG-SIMPLE-CYCLE is **NP**-complete.

Question 34.5-8

Half-3-CNF-SAT asks whether an assignment makes exactly half the clauses true and half false. Prove it is NP-complete.

Answer

In NP: evaluate the clauses and check that exactly half are true.

Reduce from 3-Cnf-Sat: if ϕ has m clauses, introduce y and let

$$T = (y \vee \neg y \vee y), \quad F = (y \vee y \vee y), \quad \phi' = \phi \wedge T^m \wedge F^{2m}.$$

ϕ' has $4m$ clauses. If ϕ is satisfiable, set $y = 0$: the m clauses of ϕ and m copies of T are true, so exactly $2m$ are true. Conversely, half true is impossible with $y = 1$ because then at least $3m$ clauses are true. Thus $y = 0$; T contributes m true clauses and F contributes 0, forcing all m clauses of ϕ to be true. Therefore $\phi \in \text{3-CNF-SAT}$ iff $\phi' \in \text{HALF-3-CNF-SAT}$. The construction is polynomial, so HALF-3-CNF-SAT is **NP**-complete.